

CLINIC MANAGEMENT SAAS COMPLETE MASTER TECHNICAL SPECIFICATION

Product Vision & Objectives

Product Vision & Objectives – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
}
```

```
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
}
```

```
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
}
```

```
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
}
```



```
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Product Vision & Objectives – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",

```

```
"timestamp": "ISO"  
}  
Response:  
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Healthcare Workflow Digitization Philosophy

Healthcare Workflow Digitization Philosophy – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Healthcare Workflow Digitization Philosophy – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Healthcare Workflow Digitization Philosophy – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Healthcare Workflow Digitization Philosophy – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Healthcare Workflow Digitization Philosophy – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Healthcare Workflow Digitization Philosophy – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


User Personas & Responsibilities

User Personas & Responsibilities – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


}

User Personas & Responsibilities – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

User Personas & Responsibilities – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

User Personas & Responsibilities – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

User Personas & Responsibilities – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

User Personas & Responsibilities – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Clinic Operational Life Cycle

Clinic Operational Life Cycle – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Clinic Operational Life Cycle – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Clinic Operational Life Cycle – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Clinic Operational Life Cycle – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Clinic Operational Life Cycle – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Clinic Operational Life Cycle – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


EMR Detailed Functional Specification

EMR Detailed Functional Specification – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

EMR Detailed Functional Specification – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


}

EMR Detailed Functional Specification – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

EMR Detailed Functional Specification – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

EMR Detailed Functional Specification – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

EMR Detailed Functional Specification – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

EMR Detailed Functional Specification – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

EMR Detailed Functional Specification – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```


Appointment Engine Logic

Appointment Engine Logic – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Appointment Engine Logic – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Appointment Engine Logic – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Appointment Engine Logic – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Appointment Engine Logic – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Appointment Engine Logic – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Doctor Consultation Module

Doctor Consultation Module – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Doctor Consultation Module – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Doctor Consultation Module – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


}

Doctor Consultation Module – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Doctor Consultation Module – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Doctor Consultation Module – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Doctor Consultation Module – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Doctor Consultation Module – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```


Prescription System Rules

Prescription System Rules – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Prescription System Rules – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Prescription System Rules – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Prescription System Rules – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Prescription System Rules – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

}

Prescription System Rules – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Prescription System Rules – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Prescription System Rules – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

}

Prescription System Rules – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Prescription System Rules – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Billing & Payment Processing

Billing & Payment Processing – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Billing & Payment Processing – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Billing & Payment Processing – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Billing & Payment Processing – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Billing & Payment Processing – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

}

Billing & Payment Processing – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


}

Billing & Payment Processing – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Billing & Payment Processing – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Inventory & Pharmacy Module

Inventory & Pharmacy Module – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Inventory & Pharmacy Module – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Inventory & Pharmacy Module – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Inventory & Pharmacy Module – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Inventory & Pharmacy Module – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Inventory & Pharmacy Module – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Lab Management System

Lab Management System – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


}

Lab Management System – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Lab Management System – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Lab Management System – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Lab Management System – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Lab Management System – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Follow-up Automation Engine

Follow-up Automation Engine – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Follow-up Automation Engine – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Follow-up Automation Engine – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Follow-up Automation Engine – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Follow-up Automation Engine – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Follow-up Automation Engine – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Analytics & Reporting

Analytics & Reporting – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```

```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Analytics & Reporting – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",  
"referenceId": "UUID"  
}
```

Analytics & Reporting – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Analytics & Reporting – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Staff & Permission Matrix

Staff & Permission Matrix – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


}

Staff & Permission Matrix – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Staff & Permission Matrix – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Staff & Permission Matrix – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Staff & Permission Matrix – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Staff & Permission Matrix – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Security & Compliance Architecture

Security & Compliance Architecture – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Security & Compliance Architecture – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Security & Compliance Architecture – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Security & Compliance Architecture – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Security & Compliance Architecture – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Security & Compliance Architecture – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Multi-tenant SaaS Design

Multi-tenant SaaS Design – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Multi-tenant SaaS Design – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


}

Multi-tenant SaaS Design – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Multi-tenant SaaS Design – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Multi-tenant SaaS Design – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Multi-tenant SaaS Design – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Offline Sync Engine

Offline Sync Engine – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```



```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
```

```
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Offline Sync Engine – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema

Database Full Schema – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Database Full Schema – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Database Full Schema – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Database Full Schema – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Database Full Schema – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Database Full Schema – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


API Contract Specification

API Contract Specification – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

API Contract Specification – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


}

API Contract Specification – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

API Contract Specification – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

API Contract Specification – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

API Contract Specification – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Frontend UX Flow Per Screen

Frontend UX Flow Per Screen – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Frontend UX Flow Per Screen – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Frontend UX Flow Per Screen – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Frontend UX Flow Per Screen – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Frontend UX Flow Per Screen – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Frontend UX Flow Per Screen – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Frontend UX Flow Per Screen – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Frontend UX Flow Per Screen – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```


System Architecture

System Architecture – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```

```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
```



```
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

System Architecture – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

System Architecture – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

System Architecture – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Infrastructure & DevOps

Infrastructure & DevOps – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Infrastructure & DevOps – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


}

Infrastructure & DevOps – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Infrastructure & DevOps – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Infrastructure & DevOps – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Infrastructure & DevOps – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Scaling Strategy

Scaling Strategy – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```



```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
```

```
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Scaling Strategy – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Scaling Strategy – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Backup & Disaster Recovery

Backup & Disaster Recovery – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Backup & Disaster Recovery – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Backup & Disaster Recovery – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Backup & Disaster Recovery – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

}

Backup & Disaster Recovery – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Backup & Disaster Recovery – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Monitoring & Logging

Monitoring & Logging – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```

```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",  
"referenceId": "UUID"  
}
```

Monitoring & Logging – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Monitoring & Logging – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Monitoring & Logging – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monitoring & Logging – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy

Testing Strategy – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```

```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
```

```
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Testing Strategy – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```


Release Pipeline

Release Pipeline – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```

```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
```

```
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```



```
"status": "success",
"referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Release Pipeline – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Release Pipeline – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Roadmap

Roadmap – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```

```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
```



```
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

Roadmap – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Roadmap – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Roadmap – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Risk Management

Risk Management – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```

```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Risk Management – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Risk Management – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Risk Management – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
```

```
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

Risk Management – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Risk Management – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Risk Management – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Monetization System

Monetization System – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```

```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Monetization System – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Monetization System – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Monetization System – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
```

```
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action
Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```

```
"status": "success",
"referenceId": "UUID"
}
```

Monetization System – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Monetization System – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Monetization System – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```


Future AI Extensions

Future AI Extensions – Detailed requirement 1: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 2: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 3: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
```

```
"patientId": "UUID",
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 4: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 5: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:
POST /module/action

```
Request:
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 6: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 7: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 8: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 9: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 10: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 11: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Future AI Extensions – Detailed requirement 12: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Future AI Extensions – Detailed requirement 13: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Future AI Extensions – Detailed requirement 14: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 15: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 16: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
```

```
"data": "structured object",
"timestamp": "ISO"
}
Response:
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 17: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 18: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 19: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 20: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 21: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
```



```
"status": "success",
"referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 22: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 23: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{
  "patientId": "UUID",
  "data": "structured object",
  "timestamp": "ISO"
}
```

Response:

```
{
  "status": "success",
  "referenceId": "UUID"
}
```

Future AI Extensions – Detailed requirement 24: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```

Future AI Extensions – Detailed requirement 25: Explanation of business logic, validations, edge cases, data handling rules, performance expectations, and user interaction flow.

Data fields involved: id, reference_id, timestamps, audit flags, tenant_id, created_by, updated_by, soft_delete flag.

Failure handling: retries, rollback rules, consistency enforcement, offline queue sync.

Compliance impact: consent logs, encryption scope, access traceability.

Example API Flow:

POST /module/action

Request:

```
{  
  "patientId": "UUID",  
  "data": "structured object",  
  "timestamp": "ISO"  
}
```

Response:

```
{  
  "status": "success",  
  "referenceId": "UUID"  
}
```